SkillNavora · Business AI Stack

COMPLETE GUIDE 09 OF 15

MCP

A practical guide with steps, examples and a checklist. Read the chapter, then build the example.

AI INTEGRATION · COMPLETE GUIDE

MCP (Model Context Protocol): connecting AI assistants to your tools

MCP is an open standard that lets an AI assistant use tools and read data from external systems through one common protocol. Build an MCP server once, and any compatible assistant (Claude, Claude Code and others) can use it. This guide explains the parts and builds a small server that reads a Google Sheet of job leads.

1. The problem MCP solves

Without a standard, every AI app writes its own connector to every tool: Gmail, Sheets, Jira, your database. With MCP, a tool provider writes one server, and every MCP client can use it. The assistant asks what tools exist, chooses one, and passes arguments.

2. The three building blocks

Part	What it offers
Tools	Actions the model may call, such as `search_leads(city)` or `add_note(id, text)`. Each has a name, a description and a JSON schema for inputs.
Resources	Read-only data the client can load, such as a file or a table row, by URI.
Prompts	Reusable prompt templates the server offers to the user.

3. Architecture

```
You (chat or terminal)  ->  MCP client (Claude, Claude Code)
                              |  JSON-RPC over stdio or HTTP
                              v
                        MCP server (your code)  ->  Google Sheets / Gmail / database
```

The server runs locally (stdio) for your own machine, or remotely (HTTP) for shared tools. Start local: it is simpler and keeps your credentials on your computer.

4. Worked example: a job-leads MCP server

1. Create a folder `jobs-mcp` and install the official MCP SDK for your language (TypeScript or Python). Follow its quickstart for the exact package names, as they change.
2. Create a server with two tools: `list_leads(status)` returns rows from your sheet with that status; `set_status(row_id, status)` updates one row and allows only the values `new`, `applied`, `rejected`.
3. Give each tool a clear description. The model chooses tools by reading these descriptions, so write them as instructions: "Use to list job leads with status new before drafting applications."
4. Validate inputs on the server: reject unknown statuses; limit results to 50 rows.
5. Register the server in your client's configuration (for Claude Code, add it with the MCP command in its documentation), start a session, and ask: *"List new job leads and tell me which two fit a product role."* Watch the tool call appear and read its result.

```
# Tool definition shape (illustrative; names follow the SDK you choose)
tool "set_status":
  input:  { row_id: integer, status: "new" | "applied" | "rejected" }
  rule:   reject any other status; return the updated row
  description: "Update the status of one job lead after the user confirms."
```

5. Security

- A tool that writes (update, send, delete) should ask for confirmation in the client, or be limited to safe values, as above.
- Give the server only the permission it needs: a read-only key for read tools.
- Treat text returned by tools as data, not instructions. A web page or email can contain text that tries to steer the model.

- Log every tool call with its arguments and time.

6. Checklist

- Tools are few, named clearly, with instructions as descriptions.
- Inputs validated; writes limited or confirmed.
- Credentials kept outside code; logs enabled.
- Tested with the client, not only in isolation.

The MCP specification and SDK documentation (modelcontextprotocol.io) hold the exact message formats and current SDK names.